Write Up

By Luca Comba and Pam Savira.

Source code available at: https://github.com/lukfd/lab4_model_training

Introduction

We developed the new training pipeline and implemented the new `model/info` endpoint.

We have started by containerize the deployment of Airflow using Docker, so we wrote the `Dockerfile.fastapi` file and the `docker-compose.yml` file. That helped us developing and quickly testing our new features.

We have also dockerized the FastAPI deployment and utilized the `localstack` docker image to mock AWS S3.

We aimed at creating a *Model Factory* to make it easy to implement new Machine Learning models and integrate it with the existing Airflow Dags and Tasks. To help us with that we designed the BaseModel class, which incorporates all the basic need and high level functions of a model, like a promotion mechanism.

With a new versioning system, the `model/info` API endpoint can now return information about the model in use and some extra information about it.

Steps

New Training Pipeline DAG

We wrote a new `ml_training_pipeline_v2.py` DAG which now accept a model name as a parameter so that it can train any type of model (Cancer or Iris, depending on the Model enum).

The DAG follows a three tasks: **train** which trains the selected model using the training set, **evaluate** that uses a test set and calculates a model metrics and finally **promote** if the evaluation metrics meet the threshold, promotes the model to S3 with versioning.

```
with DAG(
    dag_id="ml_training_pipeline_v2",
    default_args=default_args,
    description="Train, evaluate, and promote ML model",
    schedule_interval=None,
    start_date=datetime(2025, 1, 1),
    catchup=False,
    params={
        "model_name": Param(
```

```
            Model.IRIS.value,
            enum=[model.value for model in Model],
            type="string",
        ),
    },
) as dag:
    # Runtime-selected model from Trigger DAG params.
    selected_model = "{{ params.model_name }}"

    train_result = train(model_name=selected_model)
    eval_result = evaluate(train_result=train_result)
    promote(eval_result=eval_result)
```

Now we have a single DAG for training, evaluate and promote multiple models.

Better Dataset

For better modularity, we created a `BaseModel` class which can be inherited from other classes, like the new `CancerModel` and backported the `IrisModel`. This was a way to standardize all models and enable the use of a `ModelFactory`.

```
class Model(Enum):
    IRIS = "iris"
    CANCER = "cancer"


class ModelFactory:
    @staticmethod
    def get_model_class(model: Model) -> Type[BaseModel]:
        if model == Model.IRIS:
            return IrisModel
        if model == Model.CANCER:
            return CancerModel
        raise ValueError(f"Unsupported model: {model}")

    @staticmethod
    def create_model(model: Model) -> BaseModel:
        return ModelFactory.get_model_class(model)()
```

This has allowed us to make the previously described `ml_training_pipeline_v2` DAG work with all models.

Model Evaluation

The model evaluation is performed using the `evaluate()` method in each model class, so it depends on the specific rules dictated by the need of the model. For example both the Iris and Cancer models uses the `accuracy_score` to measure

performance on the test set.

As requested the promotion of the model occurs only if the accuracy is greater than a threshold (default of 0.94). If the accuracy falls below this threshold, an `AirflowException` is raised and the pipeline stops.

Model Versioning

To preserve all model versions and prevent overwriting, all models created are versioned in the format `YYYYMMDDhhmmss`.

The models, if promoted, are then uploaded to S3 using the Boto library. The S3 bucket is organized by model and version.

Each model might have a helper function `get_versioned_model_dir()` that returns the appropriately versioned.

Model Promotion

Model promotion saves a promoted model to a versioned directory and uploads all artifacts to S3. The airflow promotion task then executes the promotion method of the model class that

1. Creates a versioned directory at `models/{model_name}/{datetime_version}/`
2. Copies the trained model, metadata, and metrics to the versioned directory
3. Updates `models/{model_name}/promoted_model.json` with metadata pointing to the current version
4. Uploads all versioned artifacts to S3

This helps to keep all versions and have a pointer that points to the latest version.

Model Info Endpoint

The FastAPI application exposes a `/model/info` endpoint that retrieves model metadata from S3 based on the `promoted_model.json`. The endpoint accepts an optional `model_name` query parameter to allow to get any model's metadata. The endpoint then retrieves the data from S3.

Results

We were able to successfully trigger the airflow `ml_training_pipeline_v2.py` DAG, which ran the three tasks for training, evaluating and promoting the model. As shown in the picture, the airflow web UI showed a successful run which meant that a new "cancer" model was trained, evalutated and promoted.

In this writeup we cannot show the S3 bucket as we utilized a *localstack* Docker container, which mocks all AWS S3 functionality and stores the S3 Objects in

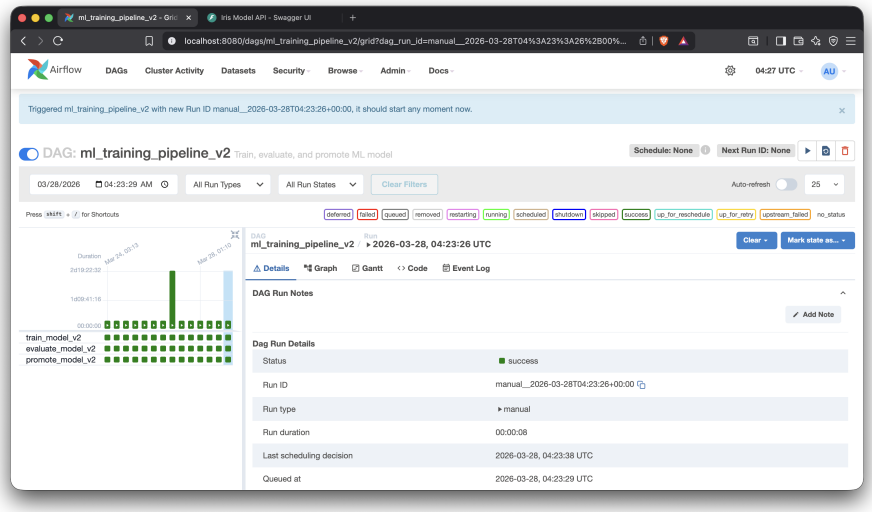

Figure 1: airflow page screenshot

memory. Therefore we are not able to take a screenshot of the objects uploaded by the promotion airflow task, but we are able to prove that it works as the `model/info` endpoint reads the metadata object from S3, and in our case the endpoint was able to return the correct data.

After the run of the DAG, the FastAPI application showed successfully the openapi.json page and the `model/info` endpoint correctly returned the metadata.json file as expected.

Short Answers

1. What problems existed in the original lab system?

- No evaluation since any model, good or bad, was immediately saved as production
- No promotion step since train equals deploy, and there is no quality gate
- No versioning since every run overwrote iris_model.pkl
- No remote storage since artifacts were local only

2. Why is storing models locally dangerous in production systems?

There is a lack of decoupling. A container restart, disk failure, or new deployment wipes the file. Multiple API instances can't share a local file. There's no audit trail when it gets overwritten.

3. Why do we add evaluation before promoting a model?

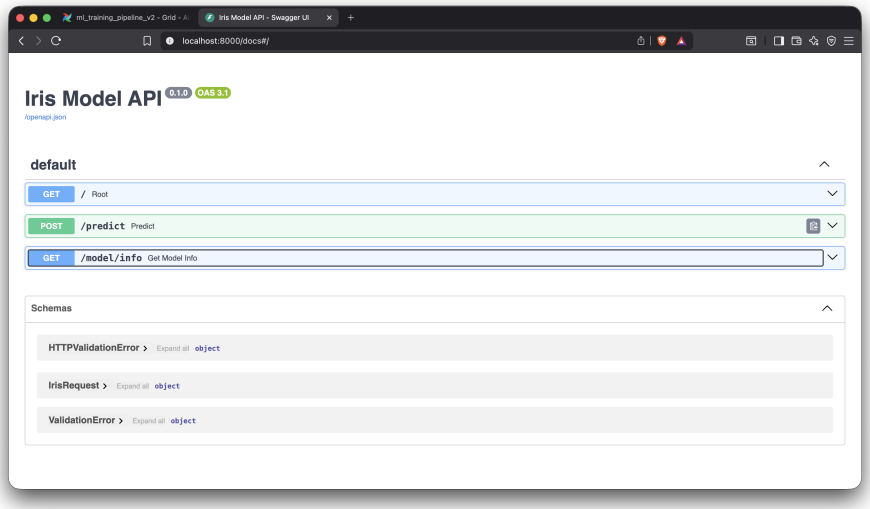

Figure 2: fastapi document page screenshot

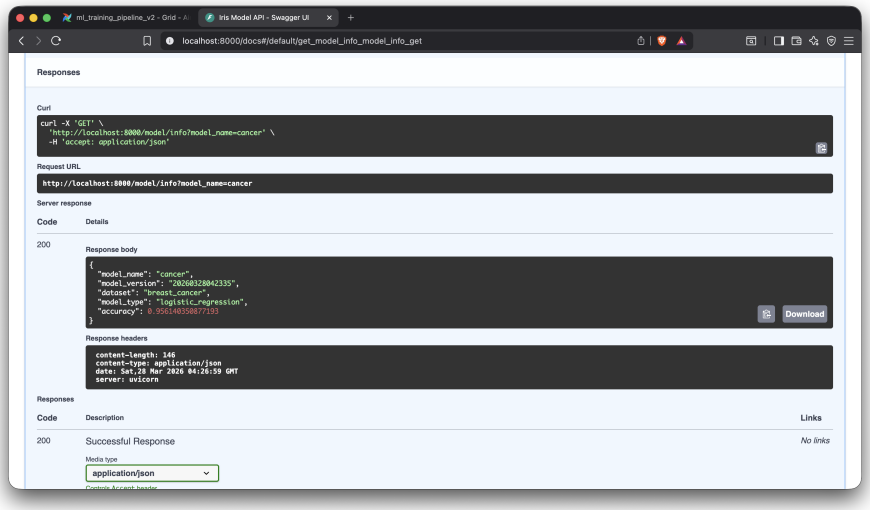

Figure 3: model info page screenshot

Training always produces a model, although it will not necessarily be a good one. The evaluation gate prevents a degraded model from reaching production; if evaluation fails, the pipeline halts and the current production model stays untouched.

4. Why do we need model versioning?

So we can roll back when a new model regresses, trace exactly what was serving during an incident, and enable A/B testing. Without versions, every promotion silently destroys the previous artifact.

5. Why might managing models manually become diffi cult as the number of models grows?

It's manageable with 2 models. With 20+ models however, we lose track of what's current vs under evaluation vs failed. Each model's thresholds and paths diverge. Deployments require manual coordination, and there's no single source of truth. Therefore, we need a proper model registry.